

Задание №4. (Командная строка, дата и время, регулярные выражения, модуль itertools)

1. В некоторой игре нужно с помощью бросков набрать N очков. За один бросок можно получить не более K очков. Сколько есть различных способов добиться цели при заданных значениях K и N ? Например, если $K=3$ и $N=4$, то существуют следующие варианты: 1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2, 1+3, 3+1. Программа должна выводить в файл различные способы и их общее количество. Значения N и K вводятся с клавиатуры. Предусмотреть запуск программы в командной строке со следующими флагами: $-N$ – значение параметра N вводится через командную строку непосредственно после данного флага (иначе, с клавиатуры), $-K$ – значение параметра K вводится через командную строку непосредственно после данного флага (иначе, с клавиатуры), $-F$ – задается имя выходного файла непосредственно после указания флага (по умолчанию файл назвать *out.txt*), флаг $-NS$ – выводить в файл только варианты (и их количество) с минимальным количеством бросков, $-t$ – записывать в файл время работы программы, $-i$ – алгоритм программы использует модуль *itertools* (по умолчанию реализовать алгоритм без использования этого модуля). Флаги могут идти в произвольном порядке. Разрешается использование модуля *argparse* для обработки аргументов командной строки.

2. Сгенерировать целочисленную матрицу N на M в виде списка. В матрице задаются два элемента (их координаты). Найти в матрице такой маршрут, соединяющий эти элементы, чтобы сумма значений элементов, встретившихся при проходе по этому маршруту, была максимальной, и вывести его. Вывести матрицу и маршрут в ней в текстовый файл (маршрут обозначить символами «*», положение элементов – «+»). Нарисовать блок-схему работы используемого алгоритма.

3. Дан лабиринт размером N на M клеток, в клетках помещены символы «S» – вход, «F» – выход, «W» – стена, «O» – проход. Двигаться от клетки к клетке можно только по вертикали и горизонтали. Построить маршрут выхода

из лабиринта. Если такового нет, то вывести соответствующее сообщение. Исходные данные в текстовом файле в виде изображения лабиринта. Выводить результат работы программы в файл: изображение поля с маршрутом, траектория которого помечена знаками «*» (или сообщение о том, что выхода нет). Программа должна сообщать любой неточности входного файла.

4. В файле имеется фрагмент кода на python, выделенный в блок между надписями «begin code» и «end code». Программа должна выполнить этот фрагмент кода. В случае ошибки в коде программа должна сообщить об этом и попросить исправить ошибку. Если таких блоков несколько, то следует выполнить второй по счету.

5. В файле имеется текст, состоящий из русских слов. Слова разделены знаком «/» или цифрой. Вывести все слова, в которых встречаются ровно три буквы «к» независимо от регистра и отсутствуют буквы «ж» и «р». При решении использовать регулярные выражения.

6. В текстовом файле заменить фрагменты вида « $p..pt..ta..a$ », где количество букв « p » ≥ 1 , « t » ≥ 0 , « a » ≥ 2 на фрагменты «xxx». При решении использовать регулярные выражения.

7. Вводится строка произвольной длины, содержащая числовые символы (например, «9999»). Имеются знаки умножения, деления, сложения, вычитания. Программа должна составить словарь с различными вариантами составления дробных и целых чисел при использовании всех символов и математических знаков («+», «-», «*», «/»). При запросе от пользователя, должна выводить все результаты вычисления заданного от пользователя числа (возможно, несокращаемой рациональной дроби). Например, словарь: {«9+9+9+9»:36: «99+9+9»:117: «9-9+9*9»: 81 и т.д.}. Запрос от пользователя: 1/111, вывод: 1/111=9/999. Чтобы не было ошибок округления – используйте модуль *fractions*.